WEB APPLICATION ARCHITECTURE

From User Request to Server Response Client–Server Architecture | HTTP/HTTPS | Database Connectivity | Web Security

> *"What happens behind the screen when a user interacts with a web application?"*

1 The web application problem

A web application must:
- accept user input and process requests
- store and retrieve data
- talk between client and server
- authenticate users when required
- return useful results, over a secure connection

How do all these parts work as one system?

Client–server | HTTP/HTTPS | Server-side processing | Database connectivity | Security

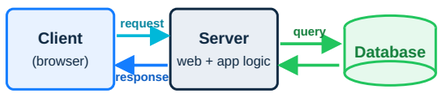
2 What is a web application?

A program that runs across a browser and a server, used through a web page.

User interface | Client-side logic | Server-side processing | Data storage | Communication | Security

Examples only: online banking, e-commerce, college portal, social media, booking, learning platform.

3 Client–server architecture



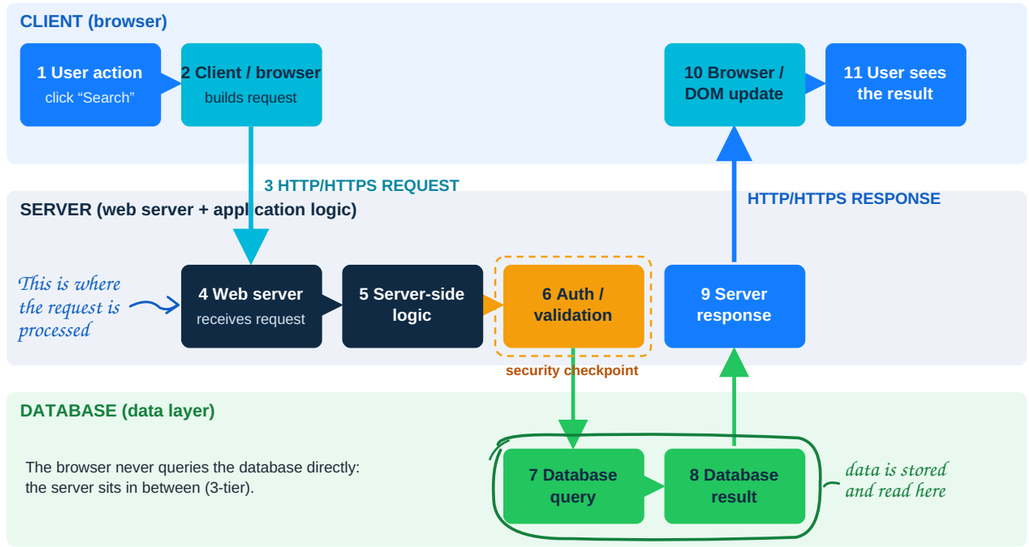
Client: requests services, shows results. **Server:** processes requests, sends responses. **Database:** stores and retrieves data.

4 How HTTP / HTTPS works

HTTP = HyperText Transfer Protocol, the rules for requests and responses. **HTTPS** = HTTP with encryption (TLS).
Caution: HTTPS protects data in transit only. A secure app also needs validation, authentication, authorization and safe sessions.

11 Complete web application flow

Client / user | Communication | Server logic | Database / data | Security



5 Request and response sequence

User → Browser → Server → Database → Server → Browser → User

1 Click Search → 2 Request sent → 3 Request processed → 4 Data queried → 5 Data returned → 6 Response built → 7 Results appear

6 Frontend / client-side

HTML: structure | CSS: presentation | JavaScript: behaviour

DOM represents the page as objects; events detect user actions.

Click → Listener → Function → DOM update

```
button.addEventListener("click", () => {
  // handle user action
});
```

7 Server-side processing

Request → Validate → Business logic → Database? → Build response → Send

Uses functions, form handling, sessions, cookies and OOP (classes, objects). Explained conceptually, not tied to one language.

9 Authentication, sessions, security

Login request → Verify credentials → Create session → Authorized access → Logout → Session ends

Authentication = who are you? **Authorization** = what may you access?
- HTTPS, input validation, hashed passwords
- Safe session management, proper error handling
- Protect against common attacks

8 Database connectivity

App → Connection → Query → Result → App

id	name	city
101	(row = one record)	
102	columns = fields	

INSERT | SELECT | UPDATE | DELETE | WHERE | JOIN

```
SELECT name FROM Students WHERE id = 101;
```

SQL is an example for relational databases. Models: Relational | NoSQL | Key-value | Document.

10 2-tier vs 3-tier

2-tier

Client ↕ Database

3-tier

Presentation ↓ Business logic ↓ Data layer

Layers improve maintainability, separation of concerns, scalability, organisation and security.

12 Example: user searches a product

Search → DOM event → HTTP request → Server → DB query → Response → UI update

```
fetch("/search?q=book");   // client
GET /search                // server
SELECT ... WHERE ...       // database
```

Illustration only, not an application project.

13–14 Workflow and real world

Plan → Design UI → Build client → Events → Server logic → Database → Security → Test → Deploy

Design → Development → Testing → Deployment → Maintenance

Same architecture, different applications

Banking | E-commerce | College portal | Healthcare | Social media | Learning | Booking

Features differ; request → processing → data → response stays similar.

15 Design considerations

UI: usability, accessibility, responsive, clear navigation
Server: validation, errors, authn, authz
Database: right type, integrity, efficient queries
Security: HTTPS, sessions, input validation
Testing: functional, security, performance

OUR ANALYSIS *what we understood*

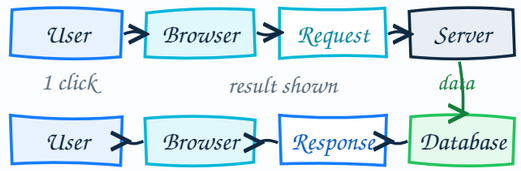
OUR OBSERVATION

We observed that a web application is not just the webpage we see. Different parts like the browser, server and database work together in the background to give us the required result.

OUR UNDERSTANDING

When we interact with a website, our action creates a request. The request goes to the server, the server processes it and gets data when needed, and then sends a response back to the browser.

OUR DIAGRAM



OUR EXAMPLE

A simple example we found is online banking. When a user checks their account balance, the browser sends a request to the server. After verification, the server retrieves the required information and sends the result back to the user.

WHAT WE LEARNED

We learned that good web application architecture connects the user interface, communication, server processing, database and security. Understanding the request–response cycle helped us understand what happens behind a website when we use it.